TASK 3. : Hand the team a corrected Query

1.First as a subquery using IN or NOT IN.

```sql
-- Task 3.3
-- 1 First as a subquery using IN or NOT IN.
SELECT DISTINCT rider_id  372 ms
FROM trips
WHERE rider_id IN (
    SELECT rider_id FROM trips WHERE trip_status = 'disputed'
);
```

Output | 1 First as a subquer...y using IN or NOT IN. | Tx: Auto | DDL

	rider_id
1	9
2	24
3	25
4	3
5	1
6	5
7	16

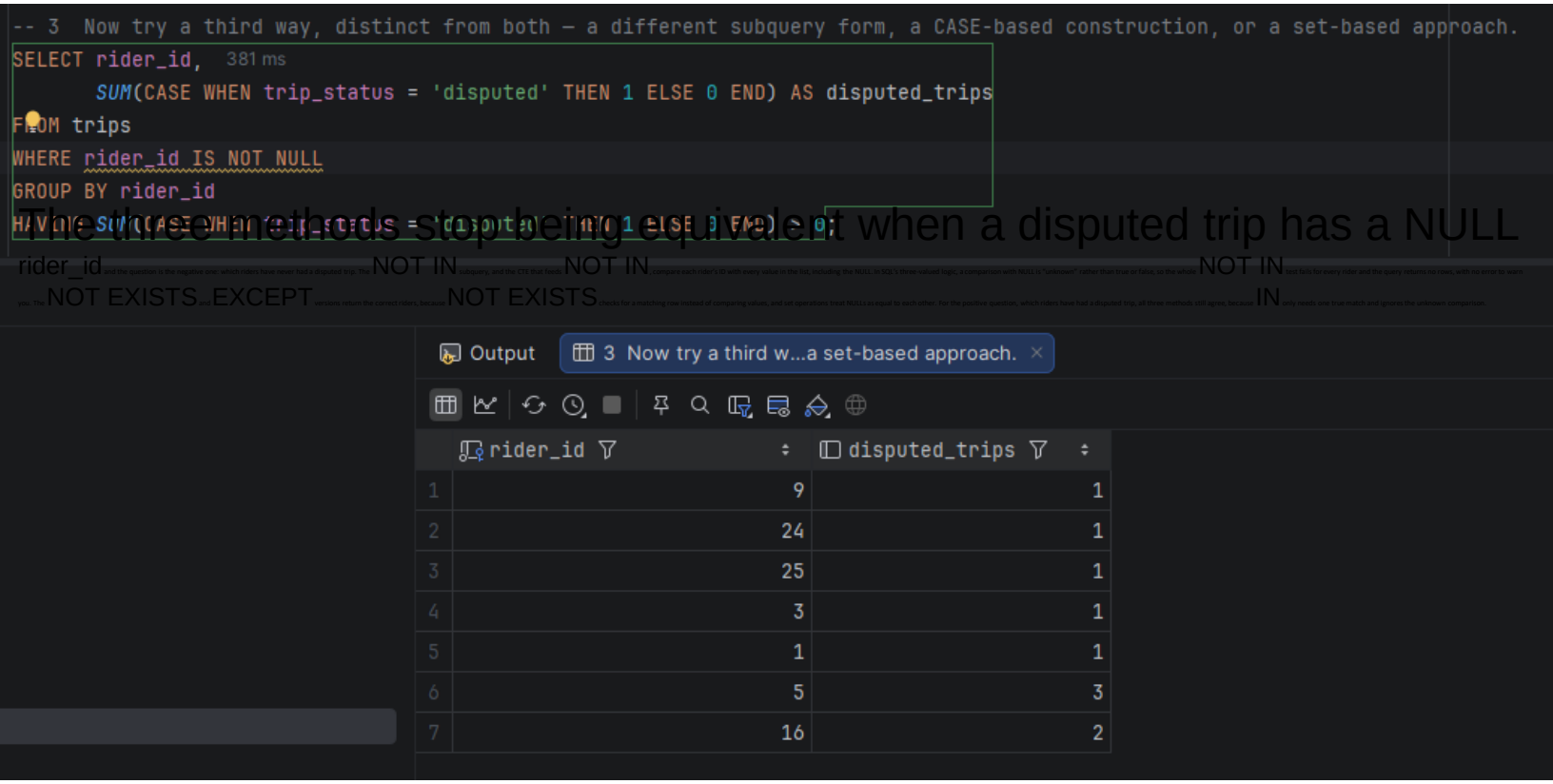
2. Then as a Common Table Expression using WITH.

```sql
-- 2 Then as a Common Table Expression using WITH.
WITH disputed_riders AS (  356 ms
    SELECT DISTINCT rider_id
    FROM trips
    WHERE trip_status = 'disputed'
)
SELECT rider_id
FROM disputed_riders
ORDER BY rider_id;
```

Output | 2 Then as a Common T...xp

	rider_id
1	1
2	3
3	5
4	9
5	16
6	24
7	25

3. Then a third way, distinct from both — a different subquery form, a CASE-based construction, or a set-based approach.



4. Demonstrate that all three return identical result sets. Matching row counts is not enough: show that the rows themselves match.

The results show that each method identified that riders 1, 3, 5, 9, 16, 24, 25 were involved in disputes

5. Name one input condition under which the three would not be equivalent. NULLs, duplicates, and empty result sets are the usual places equivalence breaks.

The three methods stop being equivalent when a disputed trip has a NULL rider_id and the question is the negative one: which riders have never had a disputed trip. The NOT IN subquery, and the CTE that feeds NOT IN, compare each rider's ID with every value in the list, including the NULL. In SQL's three-valued logic, a comparison with NULL is "unknown" rather than true or false, so the whole NOT IN test fails for every rider and the query returns no rows, with no error to warn you. The NOT EXISTS and EXCEPT versions return the correct riders, because NOT EXISTS checks for a matching row instead of comparing values, and set operations treat NULLs as equal to each other. For the positive question, which riders have had a disputed trip, all three methods still agree, because IN only needs one true match and ignores the unknown comparison.

The three methods stop being equivalent when a disputed trip has a NULL rider_id and the question is the negative one: which riders have never had a disputed trip. The NOT IN subquery, and the CTE that feeds NOT IN, compare each rider's ID with every value in the list, including the NULL. In SQL's three-valued logic, a comparison with NULL is "unknown" rather than true or false, so the whole NOT IN test fails for every rider and the query returns no rows, with no error to warn you. The NOT EXISTS and EXCEPT versions return the correct riders, because NOT EXISTS checks for a matching row instead of comparing values, and set operations treat NULLs as equal to each other. For the positive question, which riders have had a disputed trip, all three methods still agree, because IN only needs one true match and ignores the unknown comparison.